ORACLE

Java Essential

Суперклас Object

ITVDN
IT VIDEO DEVELOPERS NETWORK

Інформаційний відеосервіс для розробників програмного забезпечення

Після уроку обов'язково

Повторіть цей урок у форматі відео на ITVDN.com

Доступ можна отримати через керівництво вашого навчального центру

Перевірте, як Ви засвоїли цей матеріал на TestProvider.com

Суперклас Object

Суперклас Object

Object – це базовий клас для всіх інших об'єктів у Java. Кожен клас успадковується від Object. Відповідно всі класи успадковують методи класу Object.

Методи класу Object

public final native Class getClass() повертає об'єкт класу Class, який описує клас (ім'я, методи, поля), від якого був породжений цей об'єкт.

public native int hashCode() повертає значення int. Мета хеш-коду – представити будь-який об'єкт цілим числом. Необхідно, щоб об'єкти, рівні за значенням, повертали однакові хеш-коди.

public boolean equals (Object obj) порівнює об'єкти за значенням, а не за посиланням. Порівнюється стан об'єкта, у якого викликається цей метод, з аргументом, що передається.

public String toString() дає змогу отримати текстовий опис будь-якого об'єкта. Створюючи новий клас, цей метод можна перевизначити та повертати докладніший опис. Для класу Object і його спадкоємців, які не визначили toString(), метод повертає такий вираз:

```
getClass().getName()+"@"+hashCode()
```

Методи класу Object

protected native Object clone() throws CloneNotSupportedException – під час виконання методу clone() спочатку перевіряється, чи можна клонувати вихідний об'єкт. Якщо розробник хоче зробити об'єкти свого класу доступними для клонування через Object.clone(), він має реалізувати у своєму класі інтерфейс Cloneable. У цьому інтерфейсі немає жодного елемента, він є лише ознакою для віртуальної машини, що об'єкти можуть бути клоновані. В іншому випадку метод породжує помилку CloneNotSupportedException.

Якщо інтерфейс Cloneable реалізований, то породжується новий об'єкт від того самого класу, від якого було створено вихідний об'єкт. Водночас копіювання виконується на рівні віртуальної машини, ніякі конструктори не викликаються. Потім значення всіх полів, оголошених, успадкованих чи оголошених у батьківських класах, копіюються. Отриманий об'єкт повертається як клон.

Примітивні поля копіюються і надалі існують незалежно у вихідному та клонованому об'єктах. Зміна одного не впливає на інший.

Посилальні поля копіюються за посиланням, обидва об'єкти посилаються на ту саму область пам'яті (початковий об'єкт). Тому зміни, що відбуваються з вихідним об'єктом, впливають на клонований.

Методи класу Object

public final native void notify() відновлює виконання потоку, який очікує об'єкта, що викликає.

public final native void notifyAll() відновлює виконання всіх потоків, які очікують об'єкта, що викликає.

public final void wait() throws InterruptedException переводить виконання потоку в режим очікування, доки не буде викликаний метод notify() або notifyAll().

public final native void wait (long timeout) throws InterruptedException – перевантажений метод. Переводить виконання потоку в режим очікування, доки не буде викликаний метод notify() чи notifyAll(), чи доки не закінчиться timeout, заданий у мілісекундах.

public final void wait (long timeout, int nanos) throws InterruptedException – перевантажений метод. Переводить виконання потоку в режим очікування, доки не буде викликаний метод notify() або доки не закінчиться проміжок часу timeout+nanos, заданий в мілісекундах і наносекундах відповідно.

Методи класу Object

protected void finalize() throws Throwable – цей метод викликається під час знищення об'єкта автоматичним збирачем сміття (garbage collector). У класі Object він нічого не робить, однак у класі-спадкоємці дозволяє описати всі дії, які необхідні для коректного видалення об'єкта, як-от закриття з'єднань із БД, мережевих з'єднань, зняття блокувань на файли тощо. У звичайному режимі напряму цей метод викликати не потрібно, він відпрацює автоматично. Якщо потрібно, можна звернутися до нього явно.

У методі finalize() варто описувати лише додаткові дії, які пов'язані з логікою роботи програми. Усе необхідне для видалення об'єкта JVM зробить сама.

Модифікатор static

- Застосовується до внутрішніх класів, методів, змінних і логічних блоків.
- Статичні змінні ініціалізуються під час завантаження класу.
- Статичні змінні єдині для всіх об'єктів класу (однакове посилання).
- Статичні методи мають доступ лише до статичних змінних.
- До статичних методів і змінних можна звертатися через ім'я класу.
- Статичні блоки виконуються під час завантаження класу.
- Не static-методи не можуть бути перевизначені як static.
- Локальні змінні не можуть бути оголошені як static.
- Абстрактні методи не можуть бути static.
- Static-поля не серіалізуються (тільки під час реалізації інтерфейсу Serializable).
- Тільки static-змінні класу можуть бути передані в конструктор з параметрами, що викликається через слово super(//параметр//) або this(//параметр//).

Модифікатор abstract

1. Застосовується лише для методів і класів.
2. Абстрактні методи не мають тіла методу.
3. Є протилежністю final: final-клас не може успадковуватися, abstract-клас має успадковуватися.

Клас має бути оголошений як abstract якщо:

- він містить хоч би один абстрактний метод;
- він не надає реалізацію успадкованим абстрактним методам;
- не надає реалізацію методів інтерфейсу, реалізацію якого він оголосив;
- необхідно заборонити створення екземплярів класу.

Модифікатор final

- Поля не можуть бути зміненими, методи перевизначені.
- Класи не можна успадковувати.
- Цей модифікатор застосовується лише до класів, методів та змінних (також і до локальних змінних).
- Аргументи методів, які позначені як final, призначені лише для читання. Під час спроби зміни буде помилка компіляції.
- Змінні final не ініціалізуються за замовчуванням, їм необхідно явно надати значення під час оголошення або в конструкторі, інакше – помилка компіляції.
- Якщо final-змінна містить посилання на об'єкт, об'єкт може бути змінений, але змінна завжди буде посилатися на той самий об'єкт.
- Також це справедливо і для масивів, бо масиви є об'єктами, – масив може бути змінений, а змінна завжди буде посилатися на той самий масив.
- Якщо клас оголошено final і abstract (взаємозаперечні поняття), буде помилка компіляції.
- Оскільки final-клас не може успадковуватись, його методи ніколи не можуть бути перевизначені.

Модифікатор strictfp

- Застосовується для методів і класів.
- Забезпечує виконання операцій над числами типу **float** і **double** (з рухомою комою) за стандартом **IEEE 754**.

Модифікатор transient

- Застосовується тільки для змінних рівня класу (локальні змінні не можуть бути оголошені як transient).
- Transient-змінні можуть не бути final або static.
- Transient-змінні не серіалізуються.

Модифікатор volatile

- Використовується лише зі змінними.
- Може використовуватись зі static-змінними.
- Не використовується з final-змінними – значення змінної, оголошеної як volatile, змінене одним потоком, асинхронно змінюється для інших потоків.
- Застосовується у багатопотокових застосунках.

Модифікатор synchronized

- Застосовується лише до методів чи частин методів.
- Використовується для контролю доступу до важливих частин коду у багатопотокових програмах.

Модифікатор native

- Використовується лише для методів.
- Окреслює, що метод написаний іншою мовою програмування.
- Класи в Java використовують багато native методів для підвищення продуктивності та доступу до апаратних засобів.
- Можна передавати/повертати Java-об'єкти з native-методів.
- Сигнатура методу має закінчуватися «;», фігурні дужки спричинять помилку компіляції.

Модифікатор interface

- Методи завжди **public** і **abstract**, навіть якщо це не оголошено.
- Методи не можуть бути **final, strictfp, native, private, protected**.
- Змінні тільки **public static final**, навіть якщо це не оголошено.
- Змінні не можуть бути **strictfp, native, private, protected**.
- Може лише успадковувати (**extends**) інший інтерфейс, але не реалізовувати інтерфейс чи клас (**implements**).

Ключові слова

Зведена таблиця

	Клас	Внутрішній клас	Змінна	Метод	Конструктор	Логічний блок
public	Так	Так (окрім локальних та анонімних класів)	Так	Так	Так	Ні
protected	Ні	Так (окрім локальних та анонімних класів)	Так	Так	Так	Ні
default	Ні	Так	Так (і для локальної змінної)	Так	Так	Так
private	Ні	Так (окрім локальних та анонімних класів)	Так	Так	Так	Ні
final	Так	Так (окрім анонімних класів)	Так (і для локальної змінної)	Так	Ні	Ні
abstract	Так	Так (окрім анонімних класів)	Ні	Так	Ні	Ні

Ключові слова

Зведена таблиця

	Клас	Внутрішній клас	Змінна	Метод	Конструктор	Логічний блок
static	Ні	Так (окрім локальних та анонімних класів)	Так	Так	Ні	Так
native	Ні	Ні	Ні	Так	Ні	Ні
transient	Ні	Ні	Так	Ні	Ні	Ні
synchronized	Ні	Ні	Ні	Так	Ні	Так (тільки як частина методу)
volatile	Ні	Ні	Так	Ні	Ні	Ні
strictfp	Так	Так	Ні	Так	Ні	Ні

Q&A

ITVDN

IT VIDEO DEVELOPERS NETWORK